46-SVPWM 17

大家晚上好，欢迎参加张飞实战电子 FOC 第 11 期学习。

上节课我们讲了根据霍尔信号求转子位置角的代码。下面，我们来梳理一下整个程序逻辑啊。首先呢，我们会把需要使用的外设进行初始化。

```
/*==========*/
void StartProgram(void)
{
    unsigned int Temp;

    ENIE();
    Initialize();
    MainDelayTime = 20000; //1s

    do
    {
        Temp = MainDelayTime;
    }while(Temp != 0);

    while(1)
    {
        BldcApp();
        ProtectApp();
        LedApp(tProtect.Code);
    }
}
```

整个初始化呢，就通过调用这个 Initialize()函数来实现。

```
/*==========*/
void Initialize(void)
{
    McuInitialize();

    BldcInit();
}
/*==========*/
//函数功能：单片机底层初始化函数
//函数输入：无
//函数输出：无
//作者：张飞实战电子
/*==========*/
static void McuInitialize(void)
{
    McuPortAInitialize();
    McuPortBInitialize();
    McuPortCInitialize();
    McuPpsSelectInitialize();
    McuTimer3Initialize();
    McuIcInitialize();
    McuPwmInitialize();
    McuAdcInitialize();
    McuSpiInitialize();
    McuIntInitialize();
}
```

这个初始化函数中，包含了单片机底层外设模块的初始化以及应用代码的初始化。底层，包含 IO 端口的初始化以及其它的诸如 PWM，ADC，SPI 等外设的初始化。也就是对相关外设的寄存器进行配置。我们会采用 PWM 模块来触发 ADC 转换。然后，当 ADC 转换完成后，就会进

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

入 AD 中断。我们所有的电机控制的核心任务，都会在这个 AD 中断中执行。而我们设置的 PWM 模块的载频是 20KHz，所以这个 AD 中断也就是 1/20KHz = 50us 响应一次。

```
void StartProgram(void)
{
    unsigned int Temp;

    ENIE();
    Initialize();
    MainDelayTime = 20000; //1s

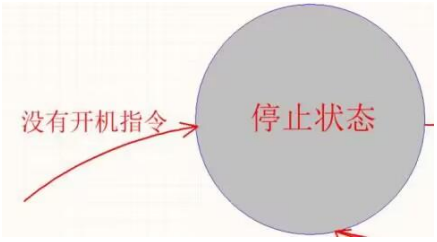
    do
    {
        Temp = MainDelayTime;
    }while(Temp != 0);

    while(1)
    {
        BldcApp();
        ProtectApp();
        LedApp(tProtect.Code);
    }
}
/*=====================================================================*/
//50us Interrupt
void __attribute__((section(".main_section"),interrupt,shadow,no_auto_psv)) _AD1Interrupt(void)
{
    IFS0bits.AD1IF = 0;
    BldcInt();
    LedInt();
    ProtectInt();
    SpiTransmit();
    if(MainDelayTime)
        MainDelayTime --;
}
```

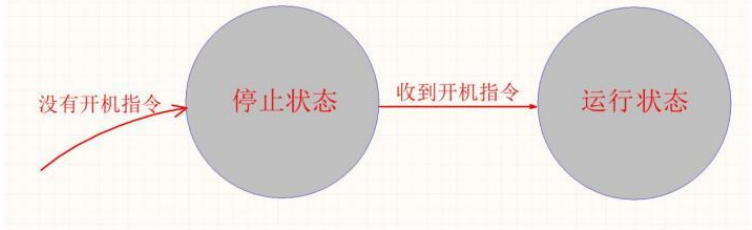
所以，我们程序的逻辑架构呢，主要包含主函数及一个中断函数。中断函数，每 50us 响应一次，其余时间都会执行主循环中的函数。实时性要求比较高的任务，就放在中断中执行，没有实时性要求，或者要求比较低的，就在主循环中执行。这个呢，就是我们程序逻辑的大框架。

然后，现在我们重点来看中断中的任务。大家还需要明确的是，实际上整个控制器，它会遵循一个状态机执行。也就是说，我们的电机，并不是控制器一上电，就自己转动。它还是需要一些控制信号的。

比如说，一个开关，当我们按下开关的时候，才会让电机转动，没有按开关的时候呢，电机是停止的。所以啊，电机的运转，其实就涉及到一些状态跳转。比如说，初始的状态是停止状态，然后呢，我们按下开关，就会跳转到其它的状态。

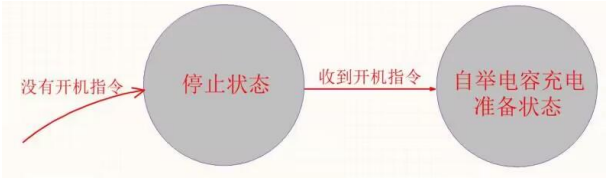


大家看，假设我们用状态图画一下，此时系统的状态就处于停止状态。



本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

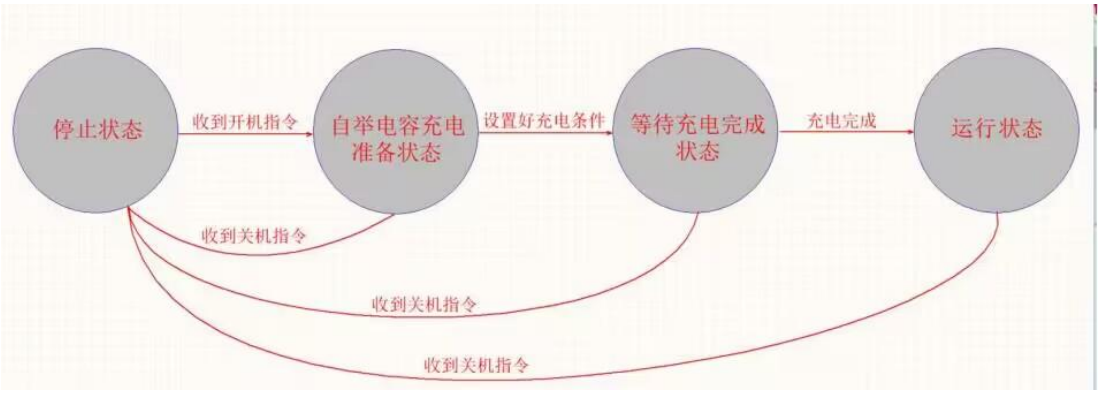
然后，当收到开机指令，就让状态跳转到运行状态，然后让电机开始转动。但是呢，实际上在停止状态跟运行状态之间，往往还会增加其它一些状态。例如，自举电容充电设置状态、等待自举电容充电完成状态等。给自举电容充电的原因是因为，三相逆变桥中的上管要想正常工作，就需要有相应的能量，而这个能量是由自举电容提供的。所以啊，在电机正常转动之前呢，我们往往会先对自举电容进行充电。



所以，一般来说，当在停止状态接收到开机指令后，会先跳转到自举电容充电准备状态而这个准备呢，无非就是打开三个下管，然后再设定好充电的时间。这个设置好之后呢，就会跳转到等待充电完成状态。在这个状态呢，当等到充电完成后，也会做一些初始化的工作例如，此时我们可以读霍尔的状态，确定转子的初始位置，为后面电机正常运转做准备。



所以呢，大家看下，我们这里的状态机呢，大致就包含这四个状态。当然了，如果在其它状态的时候，我们收到关机指令，也会跳转到停止状态。



所以，状态图可以变为这样。因此啊，我们需要根据这个状态图，来写一个状态跳转函数。这个函数呢，我们也称为状态机函数。

问：咱们这个互不 PWM 还需要自举电容充电吗？

答：建议充电，至少可以保证前面几个脉冲可以正常工作，欠压状态工作，对功率管也不是很友好。

问：自举电容充满的条件是什么？上 N 管能正常工作吗，还是说用 t=rc 计算充电时间进行延时？

答：>10V，工程上采用延时，通过示波器观测自举电容充电需要多长时间。

下面我们一起来看下状态机对应的代码。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

```
/*=====================================================================*/
//状态控制函数
static void BldcStatusControl(void)
{
    if(tBldc.DownCount)
        tBldc.DownCount --;

    if(tBldc.Staus == 0) //0:Stop Status
    {
        IOCON1 = IOCON_HL_INVALID;
        IOCON2 = IOCON_HL_INVALID;
        IOCON3 = IOCON_HL_INVALID;
    }
    else if(tBldc.Staus == 1) //1:BootCharge Init
    {
        IOCON1 = IOCON_HINVALID_LPWM;
        IOCON2 = IOCON_HINVALID_LPWM;
        IOCON3 = IOCON_HINVALID_LPWM;
        tBldc.DownCount = 2000;
        tBldc.Staus = 2;
    }
    else if(tBldc.Staus == 2)//2:BootCharge Finish
    {
        if(tBldc.DownCount == 0)
        {
            IOCON1,IOCON_HL_INVALID;
```

大家看下，这段程序，表示的就是状态跳转的流程

```
/*=====================================================================*/
//状态控制函数
static void BldcStatusControl(void)
{
    if(tBldc.DownCount)
        tBldc.DownCount --;

    if(tBldc.Staus == 0) //0:Stop Status
    {
        IOCON1 = IOCON_HL_INVALID;
        IOCON2 = IOCON_HL_INVALID;
        IOCON3 = IOCON_HL_INVALID;
    }
    else if(tBldc.Staus == 1) //1:BootCharge Init
    {
        IOCON1 = IOCON_HINVALID_LPWM;
        IOCON2 = IOCON_HINVALID_LPWM;
        IOCON3 = IOCON_HINVALID_LPWM;
        tBldc.DownCount = 2000;
        tBldc.Staus = 2;
    }
```

Status 是一个状态变量，它等于 0 的时候，就对应停止状态。在这个状态下呢，它不会自动跳转到其它的状态，而是需要等待接收一个开机指令。只有收到开机指令，才会从停止状态跳转到其它的状态。那么，在停止状态，执行什么任务呢？

```
IOCON1 = IOCON_HL_INVALID;
IOCON2 = IOCON_HL_INVALID;
IOCON3 = IOCON_HL_INVALID;
```

任务，就是这三句话，也就是分别对 IOCON1/2/3 进行赋值。

```
/*=====================================================================*/
#define     IOCON_HL_INVALID        0xc300
#define     IOCON_HINVALID_LPWM     0xc200
#define     IOCON_HPWM_LPWM         0xc000
/*=====================================================================*/
```

赋的值呢，对应一个宏定义值，也就是对 IOCON1/2/3 赋值 0xc300 。这个 0xc300 表示什么意思呢？此时，我们就需要查对应的手册了。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

大家打开手册，查一下，看看是啥意思啊？

寄存器 16-13：　IOCONx：PWMx I/O 控制寄存器 (2)

R/W-1	R/W-1	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0
PENH	PENL	POLH	POLL	PMOD<1:0>(1)		OVRENH	OVRENL
bit 15							bit 8

R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0
OVRDAT<1:0>		FLTDAT<1:0>		CLDAT<1:0>		SWAP	OSYNC
bit 7							bit 0

图注：			
R = 可读位	W = 可写位	U = 未实现位，读为 0	
-n = POR 时的值	1 = 置 1	0 = 清零	x = 未知

bit 15　**PENH**：PWMxH 输出引脚所有权位
1 = PWMx 模块控制 PWMxH 引脚
0 = GPIO 模块控制 PWMxH 引脚

bit 14　**PENL**：PWMxL 输出引脚所有权位
1 = PWMx 模块控制 PWMxL 引脚
0 = GPIO 模块控制 PWMxL 引脚

bit 13　**POLH**：PWMxH 输出引脚极性位
1 = PWMxH 引脚为低电平有效
0 = PWMxH 引脚为高电平有效

bit 12　**POLL**：PWMxL 输出引脚极性位
1 = PWMxL 引脚为低电平有效
0 = PWMxL 引脚为高电平有效

bit 11-10　**PMOD<1:0>**：PWMx # I/O 引脚模式位 (1)
11 = 保留：不要使用
10 = PWMx I/O 引脚对处于推挽输出模式
01 = PWMx I/O 引脚对处于冗余输出模式
00 = PWMx I/O 引脚对处于互补输出模式

bit 9　**OVRENH**：PWMxH 引脚改写使能位
1 = OVRDAT<1> 控制 PWMxH 引脚上的输出
0 = PWMx 发生器控制 PWMxH 引脚

bit 8　**OVRENL**：PWMxL 引脚改写使能位

打开手册一查，其实我们就可以知道，这样赋值，其实就是对 6 个 PWM 引脚强制输出为 0 也即是说，在停止状态，我们就是保持 PWM 引脚始终输出低电平。

```
//状态控制函数
static void BldcStatusControl(void)
{
    if(tBldc.DownCount)
        tBldc.DownCount --;

    if(tBldc.Staus == 0) //0:Stop Status
    {
        IOCON1 = IOCON_HL_INVALID;
        IOCON2 = IOCON_HL_INVALID;
        IOCON3 = IOCON_HL_INVALID;
    }
    else if(tBldc.Staus == 1) //1:BootCharge Init
    {
        IOCON1 = IOCON_HINVALID_LPWM;
        IOCON2 = IOCON_HINVALID_LPWM;
        IOCON3 = IOCON_HINVALID_LPWM;
        tBldc.DownCount = 2000;
        tBldc.Staus = 2;
    }
    else if(tBldc.Staus == 2)//2:BootCharge Finish
    {
        if(tBldc.DownCount == 0)
        {
            IOCON1, IOCON_HL_INVALID;
            IOCON2, IOCON_HL_INVALID;
            IOCON3, IOCON_HL_INVALID;
            HallInitialize();
            tBldc.Staus = 3;
        }
    }
```


本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

那么，如果在停止状态，接收到了开机指令，状态才会跳转到下面的自举电容充电准备状态。

```
//状态控制函数
static void BldcStatusControl(void)
{
    if(tBldc.DownCount)
        tBldc.DownCount --;

    if(tBldc.Staus == 0) //0:Stop Status
    {
        IOCON1 = IOCON_HL_INVALID;
        IOCON2 = IOCON_HL_INVALID;
        IOCON3 = IOCON_HL_INVALID;
    }
    else if(tBldc.Staus == 1) //1:BootCharge Init
    {
        IOCON1 = IOCON_HINVALID_LPWM;
        IOCON2 = IOCON_HINVALID_LPWM;
        IOCON3 = IOCON_HINVALID_LPWM;
        tBldc.DownCount = 2000;
        tBldc.Staus = 2;
    }
    else if(tBldc.Staus == 2)//2:BootCharge Finish
    {
        if(tBldc.DownCount == 0)
        {
            IOCON1, IOCON_HL_INVALID;
            IOCON2, IOCON_HL_INVALID;
            IOCON3, IOCON_HL_INVALID;
            HallInitialize();
            tBldc.Staus = 3;
        }
    }
```

在这个状态呢，主要完成两件事。一是让下管打开，或者说让下管的开通占空比大一些。二是设置充电时间。我们这里呢，就是使用下管开通占空比大一些的方式来充电。

```
#define     IOCON_HL_INVALID        0xc300
#define     IOCON_HINVALID_LPWM     0xc200
#define     IOCON_HPWM_LPWM         0xc000
```

此时，对应的值是 0xc200。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

R/W-1	R/W-1	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0
PENH	PENL	POLH	POLL	PMOD<1:0>(1)		OVRENH	OVRENL
bit 15							bit 8

R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0
OVRDAT<1:0>		FLTDAT<1:0>		CLDAT<1:0>		SWAP	OSYNC
bit 7							bit 0

图注：			
R = 可读位	W = 可写位	U = 未实现位，读为 0	
-n = POR 时的值	1 = 置 1	0 = 清零	x = 未知

bit 15 **PENH**：PWMxH 输出引脚所有权位
1 = PWMx 模块控制 PWMxH 引脚
0 = GPIO 模块控制 PWMxH 引脚

bit 14 **PENL**：PWMxL 输出引脚所有权位
1 = PWMx 模块控制 PWMxL 引脚
0 = GPIO 模块控制 PWMxL 引脚

bit 13 **POLH**：PWMxH 输出引脚极性位
1 = PWMxH 引脚为低电平有效
0 = PWMxH 引脚为高电平有效

bit 12 **POLL**：PWMxL 输出引脚极性位
1 = PWMxL 引脚为低电平有效
0 = PWMxL 引脚为高电平有效

bit 11-10 **PMOD<1:0>**：PWMx # I/O 引脚模式位 (1)
11 = 保留；不要使用
10 = PWMx I/O 引脚对处于推挽输出模式
01 = PWMx I/O 引脚对处于冗余输出模式
00 = PWMx I/O 引脚对处于互补输出模式

bit 9 **OVRENH**：PWMxH 引脚改写使能位
1 = OVRDAT<1> 控制 PWMxH 引脚上的输出
0 = PWMx 发生器控制 PWMxH 引脚

bit 8 **OVRENL**：PWMxL 引脚改写使能位
1 = OVRDAT<0> 控制 PWMxL 引脚上的输出
0 = PWMx 发生器控制 PWMxL 引脚

bit 7-6 **OVRDAT<1:0>**：使能改写时 PWMxH 和 PWMxL 引脚数据位
如果 OVERENH = 1，则 PWMxH 被驱动为由 OVRDAT<1> 指定的状态。
如果 OVERENL = 1，则 PWMxL 被驱动为由 OVRDAT<0> 指定的状态。

这个意思就是，下管不再受强制设置为 0 了，而是由 PWM 模块来控制。下管由 PWM 发生器来控制，那么输出占空比是多少呢？

```
/*=============================================================
 PWM模块寄存器初始值宏定义
=============================================================*/
#define PTCON_INIT      0x0000
#define PTCON2_INIT     0x0000
//PWM发生器1寄存器初始值宏定义
#define PWMCON1_INIT    0x0204
#define IOCON1_INIT     0x0000
#define FCLCON1_INIT    0x0003
#define PHASE1_INIT     (PWM_PERIOD_VALUE >> 1)
#define PDC1_INIT       300
#define DTR1_INIT       0x0000
#define ALTDTR1_INIT    DEAD_TIME
#define TRGCON1_INIT    0x1000
#define TRIG1_INIT      3000
//PWM发生器2寄存器初始值宏定义
#define PWMCON2_INIT    0x0204
#define IOCON2_INIT     0x0000
#define FCLCON2_INIT    0x0003
#define PHASE2_INIT     (PWM_PERIOD_VALUE >> 1)
#define PDC2_INIT       300
#define DTR2_INIT       0x0000
#define ALTDTR2_INIT    DEAD_TIME
//PWM发生器3寄存器初始值宏定义
#define PWMCON3_INIT    0x0204
#define IOCON3_INIT     0x0000
```

此时由我们设定的这个值来决定。PDCx 寄存器，负责上管的占空比大小，而上下管是互补的，所以也就可以知道下管的占空比大小。这里上管占空比是 10%的话，那么下管就是 90%。也就是说，下管开得还是比较大的。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

```
static void BldcStatusControl(void)
{
    if(tBldc.DownCount)
        tBldc.DownCount --;

    if(tBldc.Staus == 0) //0:Stop Status
    {
        IOCON1 = IOCON_HL_INVALID;
        IOCON2 = IOCON_HL_INVALID;
        IOCON3 = IOCON_HL_INVALID;
    }
    else if(tBldc.Staus == 1) //1:BootCharge Init
    {
        IOCON1 = IOCON_HINVALID_LPWM;
        IOCON2 = IOCON_HINVALID_LPWM;
        IOCON3 = IOCON_HINVALID_LPWM;
        tBldc.DownCount = 2000;
        tBldc.Staus = 2;
    }
    else if(tBldc.Staus == 2)//2:BootCharge Finish
    {
        if(tBldc.DownCount == 0)
        {
            IOCON1, IOCON_HL_INVALID;
            IOCON2, IOCON_HL_INVALID;
            IOCON3, IOCON_HL_INVALID;
            HallInitialize();
            tBldc.Staus = 3;
        }
    }
```

所以，这三句话的设置，就是下管开 90%的占空比，给自举电容充电。

```
//状态控制函数
static void BldcStatusControl(void)
{
    if(tBldc.DownCount)
        tBldc.DownCount --;

    if(tBldc.Staus == 0) //0:Stop Status
    {
        IOCON1 = IOCON_HL_INVALID;
        IOCON2 = IOCON_HL_INVALID;
        IOCON3 = IOCON_HL_INVALID;
    }
    else if(tBldc.Staus == 1) //1:BootCharge Init
    {
        IOCON1 = IOCON_HINVALID_LPWM;
        IOCON2 = IOCON_HINVALID_LPWM;
        IOCON3 = IOCON_HINVALID_LPWM;
        tBldc.DownCount = 2000;
        tBldc.Staus = 2;
    }
    else if(tBldc.Staus == 2)//2:BootCharge Finish
    {
        if(tBldc.DownCount == 0)
        {
            IOCON1, IOCON_HL_INVALID;
```


本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

问：这时上管不是 PWM 模块控制的啊？
答：是的，充电时上管是强制关闭的。

接着呢，我们给这个计数变量设置了一个初值 2000。这个状态函数呢，会放在 50us 中断中执行，所以这个计数变量每 50us 递减一次。那么，2000 的话，其实就是 100ms。也就是说，我们打算设置自举电容的充电时间是 100ms。这样的话，自举电容充电准备状态就设置好了。

```
//状态控制函数
static void BldcStatusControl(void)
{
    if(tBldc.DownCount)
        tBldc.DownCount --;

    if(tBldc.Staus == 0) //0:Stop Status
    {
        IOCON1 = IOCON_HL_INVALID;
        IOCON2 = IOCON_HL_INVALID;
        IOCON3 = IOCON_HL_INVALID;
    }
    else if(tBldc.Staus == 1) //1:BootCharge Init
    {
        IOCON1 = IOCON_HINVALID_LPWM;
        IOCON2 = IOCON_HINVALID_LPWM;
        IOCON3 = IOCON_HINVALID_LPWM;
        tBldc.DownCount = 2000;
        tBldc.Staus = 2;
    }
    else if(tBldc.Staus == 2)//2:BootCharge Finish
    {
        if(tBldc.DownCount == 0)
        {
            IOCON1, IOCON_HL_INVALID;
            IOCON2, IOCON_HL_INVALID;
            IOCON3, IOCON_HL_INVALID;
            HallInitialize();
            tBldc.Staus = 3;
        }
    }
```

大家看，当设置好充电条件后，此时状态会自动跳转。下一个状态呢，也就是等待自举电容充电完成状态。

```
else if(tBldc.Staus == 2)//2:BootCharge Finish
{
    if(tBldc.DownCount == 0)
    {
        IOCON1 = IOCON_HL_INVALID;
        IOCON2 = IOCON_HL_INVALID;
        IOCON3 = IOCON_HL_INVALID;
        HallInitialize();
        tBldc.Staus = 3;
    }
}
```

这里，就是判断这个计数变量是否递减为 0，来看是不是充电时间到了。充电时间到了的话，就会进入到这个 if 分支内部。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

```
    else if(tBldc.Staus == 2)//2:BootCharge Finish
    {
        if(tBldc.DownCount == 0)
        {
            IOCON1 = IOCON_HL_INVALID;
            IOCON2 = IOCON_HL_INVALID;
            IOCON3 = IOCON_HL_INVALID;
            HallInitialize();
            tBldc.Staus = 3;
        }
    }
    else if(tBldc.Staus == 3) //3:Run
    {
```

此时呢，我们需要把充电状态停止掉，让 6 个管子先关闭一下。这样为后面电机转动做准备。这个宏定义我们刚才讲过，是 0xc300。

```
else if(tBldc.Staus == 2)//2:BootCharge Finish
{
    if(tBldc.DownCount == 0)
    {
        IOCON1 = IOCON_HL_INVALID;
        IOCON2 = IOCON_HL_INVALID;
        IOCON3 = IOCON_HL_INVALID;
        HallInitialize();
        tBldc.Staus = 3;
    }
}
```

同时呢，因为下一个状态就要让电机正常运转了，所以呢，这里需要先读一下静止时候的霍尔值，然后确定出当前转子的位置。

```
else if(tBldc.Staus == 2)//2:BootCharge Finish
{
    if(tBldc.DownCount == 0)
    {
        IOCON1 = IOCON_HL_INVALID;
        IOCON2 = IOCON_HL_INVALID;
        IOCON3 = IOCON_HL_INVALID;
        HallInitialize();
        tBldc.Staus = 3;
    }
}
else if(tBldc.Staus == 3) //3:Run
{
    IOCON1 = IOCON_HPWM_LPWM;
    IOCON2 = IOCON_HPWM_LPWM;
    IOCON3 = IOCON_HPWM_LPWM;

    GetRotorInfo();
    if(tBldc.Direction)
        tCore.qEAngle = tBldc.Eangle_Rotor + EANGLE90;
    else
        tCore.qEAngle = tBldc.Eangle_Rotor - EANGLE90;
    tUm.Percent = tBldc.DutyInner;
    UsOut();
}
}
```


本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

然后呢，接着状态又会自动跳转到运行状态。

```
else if(tBldc.Staus == 3) //3:Run
{
    IOCON1 = IOCON_HPWM_LPWM;
    IOCON2 = IOCON_HPWM_LPWM;
    IOCON3 = IOCON_HPWM_LPWM;

    GetRotorInfo();
    if(tBldc.Direction)
        tCore.qEAngle = tBldc.Eangle_Rotor + EANGLE90;
    else
        tCore.qEAngle = tBldc.Eangle_Rotor - EANGLE90;
    tUm.Percent = tBldc.DutyInner;
    UsOut();
}
```

到这个状态后，首先要设置上下管都由 PWM 发生器控制，对应互补模式。

```
/*=============================================================================*/
#define     IOCON_HL_INVALID        0xc300
#define     IOCON_HINVALID_LPWM     0xc200
#define     IOCON_HPWM_LPWM         0xc000
/*=============================================================================*/
```

也就是，不再有强制输出状态了，让引脚完全由 PWM 发生器来控制输出。这样的话，就可以正常输出 PWM 波了。

```
else if(tBldc.Staus == 3) //3:Run
{
    IOCON1 = IOCON_HPWM_LPWM;
    IOCON2 = IOCON_HPWM_LPWM;
    IOCON3 = IOCON_HPWM_LPWM;

    GetRotorInfo();
    if(tBldc.Direction)
        tCore.qEAngle = tBldc.Eangle_Rotor + EANGLE90;
    else
        tCore.qEAngle = tBldc.Eangle_Rotor - EANGLE90;
    tUm.Percent = tBldc.DutyInner;
    UsOut();
}
```

接着呢，就调用上次课我们分析的，读取霍尔，计算转子位置，然后根据转子位置得到电压合成矢量夹角 θ 。

```
else if(tBldc.Staus == 3) //3:Run
{
    IOCON1 = IOCON_HPWM_LPWM;
    IOCON2 = IOCON_HPWM_LPWM;
    IOCON3 = IOCON_HPWM_LPWM;

    GetRotorInfo();
    if(tBldc.Direction)
        tCore.qEAngle = tBldc.Eangle_Rotor + EANGLE90;
    else
        tCore.qEAngle = tBldc.Eangle_Rotor - EANGLE90;
    tUm.Percent = tBldc.DutyInner;
    UsOut();
}
```


本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

然后，再调用我们之前分析过的 SVPWM 模块。就可以输出 SVPWM 调制波，驱动 MOS 管，然后驱动电机运转了。后续呢，就会一直在这个状态，不断的读取霍尔值，计算转子位置角，不断更新 θ，更新 SVPWM 波，让电机连续的运转。

```
static void BldcStatusControl(void)
{
    if(tBldc.DownCount)
        tBldc.DownCount --;

    if(tBldc.Staus == 0) //0 Stop Status
    {
        IOCON1 = IOCON_HL_INVALID;
        IOCON2 = IOCON_HL_INVALID;
        IOCON3 = IOCON_HL_INVALID;
    }
    else if(tBldc.Staus == 1) //1 BootCharge Init
    {
        IOCON1 = IOCON_HINVALID_LPWM;
        IOCON2 = IOCON_HINVALID_LPWM;
        IOCON3 = IOCON_HINVALID_LPWM;
        tBldc.DownCount = 2000;
        tBldc.Staus = 2;
    }
    else if(tBldc.Staus == 2)//2 BootCharge Finish
    {
        if(tBldc.DownCount == 0)
        {
            IOCON1 = IOCON_HL_INVALID;
            IOCON2 = IOCON_HL_INVALID;
            IOCON3 = IOCON_HL_INVALID;
            HallInitialize();
            tBldc.Staus = 3;
        }
    }
    else if(tBldc.Staus == 3) //3 Run
    {
        IOCON1 = IOCON_HPWM_LPWM;
        IOCON2 = IOCON_HPWM_LPWM;
        IOCON3 = IOCON_HPWM_LPWM;

        GetRotorInfo();
        if(tBldc.Direction)
            tCore.qEAngle = tBldc.Eangle_Rotor + EANGLE90;
        else
            tCore.qEAngle = tBldc.Eangle_Rotor - EANGLE90;
        tUm.Percent = tBldc.DutyInner;
        UsOut();
    }
}
```

这就是完整的一个状态函数。大家看下，这里，可以理解的哈？

问：这种不考虑超前角的控制方式只采个母线电流做个保护就可以了吧？
答：是的，也可以人为设置一个固定的超前角。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

```
else if(tBldc.Staus == 3) //3:Run
{
    IOCON1 = IOCON_HPWM_LPWM;
    IOCON2 = IOCON_HPWM_LPWM;
    IOCON3 = IOCON_HPWM_LPWM;

    GetRotorInfo();
    if(tBldc.Direction)
        tCore.qEAngle = tBldc.Eangle_Rotor + EANGLE90;
    else
        tCore.qEAngle = tBldc.Eangle_Rotor - EANGLE90;
    tUm.Percent = tBldc.DutyInner;
    UsOut();
}
```

这里要说一下，这个 tUm.Percent 变量呢，其实就是我们之前讲过的 M。它是在 0~1 之间变化的，我们设置了它的最大值是 0.92，通过调整这个值，我们可以调整矢量圆的半径后面 tBldc.DutyInner 变量，是一个变化的值，后面我们会讲到。

```
/*=====================================
static void UsOut(void)
{
    CalSinCos(tCore.qEAngle);
    GetUsSector(tCore.qEAngle);
    GetUxyz();
    SvpwmOut();
}
```

UsOut();这个函数就是我们之前讲过的 SVPWM 模块的代码实现。包含了我们之前讲过的查表、扇区判断、计算 X　Y　Z，然后输出 SVPWM 波。

```
/*=====================================
void BldcInt(void)
{
    AdcSample();
    BldcStatusControl();
    AdcFilter();
    BldcCount();
}
```

然后，我们说状态控制函数要放到中断中调用。这个 BldcInt 函数，就是 AD 中断中执行的。

```
/*=====================================
//50us Interrupt
void __attribute__((section(".main_section"),interrupt,shadow,no_auto_psv)) _AD1Interrupt(void)
{
    IFS0bits.AD1IF = 0;
    BldcInt();
    LedInt();
    ProtectInt();
    SpiTransmit();
    if(MainDelayTime)
        MainDelayTime --;
}
/*=====Copyright belongs to ZhangFeiDianZi=====*/
```

所以，这样的话，我们就清楚了系统的状态控制流程了。但是，状态机有了，如果初始时刻系统是在停止状态，没有启动条件的话，电机也不会转动的。所以，下面我们的任务呢，就是要来给定一个启动条件。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

这个启动条件呢，对于我们的系统来说，可以是按键，也可以是电位器旋钮。按键的话，就是按下按键，来给定启动，停止条件。旋钮的话，就是旋钮值大于某个值，就给一个启动条件，小于某个值，就给停止条件。咱们这里呢，先使用旋钮来完成启动停止。后面，无感 FOC 部分呢，我们切换为按键启停控制。而这个旋钮的话，实际上对应的是一个模拟电压信号。模拟电压信号的话，也就决定了我们要使用 ADC 模块。当 PWM 触发 ADC 转换完成之后，就进入到 AD 中断，进入中断呢，我们可以读取转换完成的结果。

问：如果长时间不给启动条件，预充电会不会失效？
答：启动条件发生后，才开始预充电。



目前我们呢，仅使用了这五个模拟信号。电位器旋钮呢，右边对应的就是 AN4 的。

```
static void AdcSample(void)
{
    if(tAdc.Index == 3)
    {
        tAdc.An3.ValueInstant = ADC1BUF0;
        tAdc.An0.ValueInstant = ADC1BUF1;
        tAdc.An1.ValueInstant = ADC1BUF2;
        tAdc.An2.ValueInstant = ADC1BUF3;

        AD1CHS0 = 0x0404;
        tAdc.Index = 4;
    }
    else if(tAdc.Index == 4)
    {
        tAdc.An4.ValueInstant = ADC1BUF0;
        tAdc.An0.ValueInstant = ADC1BUF1;
        tAdc.An1.ValueInstant = ADC1BUF2;
        tAdc.An2.ValueInstant = ADC1BUF3;

        AD1CHS0 = 0x0303;
        tAdc.Index = 3;
    }
    else
    {
```

大家来看下这个函数，我们是使用 10 位 4 通道同时采样的模式。AN0　AN1　AN2 是固定的作为相电流的采集通道，所以它们是固定不变的。而剩下的 AN3　AN4 就可以使用轮询的方式来读取它们的转换结果。我们的 ADC 模块呢，有　CH0　　CH1　CH2　CH3 四个采样保持放大器，CH1　CH2　CH3 选择固定的通道，CH0 可以切换　AN3　AN4。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

```
static void AdcSample(void)
{
    if(tAdc.Index == 3)
    {
        tAdc.An3.ValueInstant = ADC1BUF0;
        tAdc.An0.ValueInstant = ADC1BUF1;
        tAdc.An1.ValueInstant = ADC1BUF2;
        tAdc.An2.ValueInstant = ADC1BUF3;

        AD1CHS0 = 0x0404;
        tAdc.Index = 4;
    }
    else if(tAdc.Index == 4)
    {
        tAdc.An4.ValueInstant = ADC1BUF0;
        tAdc.An0.ValueInstant = ADC1BUF1;
        tAdc.An1.ValueInstant = ADC1BUF2;
        tAdc.An2.ValueInstant = ADC1BUF3;

        AD1CHS0 = 0x0303;
        tAdc.Index = 3;
    }
    else
    {
```

所以大家可以看到，当 Index 是 3 的时候，CH0 对应读取的是 AN3 的转换结果，对应的结果就从 BUF0 读取。其它三个通道的，就分别从 BUF1 BUF2 BUF3 来读取，读完之后呢，要切换 CH0 的通道为 AN4。

```
static void AdcSample(void)
{
    if(tAdc.Index == 3)
    {
        tAdc.An3.ValueInstant = ADC1BUF0;
        tAdc.An0.ValueInstant = ADC1BUF1;
        tAdc.An1.ValueInstant = ADC1BUF2;
        tAdc.An2.ValueInstant = ADC1BUF3;

        AD1CHS0 = 0x0404;
        tAdc.Index = 4;
    }
    else if(tAdc.Index == 4)
    {
        tAdc.An4.ValueInstant = ADC1BUF0;
        tAdc.An0.ValueInstant = ADC1BUF1;
        tAdc.An1.ValueInstant = ADC1BUF2;
        tAdc.An2.ValueInstant = ADC1BUF3;

        AD1CHS0 = 0x0303;
        tAdc.Index = 3;
    }
```

所以呢，这两句话就是在切换转换通道。当然，AD1CHS0 这个寄存器，高 8 位可以不用配置的，因为我们只使用了 MUXA 选择的通道。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

```
static void AdcSample(void)
{
    if(tAdc.Index == 3)
    {
        tAdc.An3.ValueInstant = ADC1BUF0;
        tAdc.An0.ValueInstant = ADC1BUF1;
        tAdc.An1.ValueInstant = ADC1BUF2;
        tAdc.An2.ValueInstant = ADC1BUF3;

        AD1CHS0 = 0x0004;
        tAdc.Index = 4;
    }
    else if(tAdc.Index == 4)
    {
        tAdc.An4.ValueInstant = ADC1BUF0;
        tAdc.An0.ValueInstant = ADC1BUF1;
        tAdc.An1.ValueInstant = ADC1BUF2;
        tAdc.An2.ValueInstant = ADC1BUF3;

        AD1CHS0 = 0x0003;
        tAdc.Index = 3;
    }
    else
```

然后，我们设置了 Index 是 4 的话，下一步就会执行对应的分支。此时呢，就是读取 AN4 的转换结果。

```
static void AdcSample(void)
{
    if(tAdc.Index == 3)
    {
        tAdc.An3.ValueInstant = ADC1BUF0;
        tAdc.An0.ValueInstant = ADC1BUF1;
        tAdc.An1.ValueInstant = ADC1BUF2;
        tAdc.An2.ValueInstant = ADC1BUF3;

        AD1CHS0 = 0x0004;
        tAdc.Index = 4;
    }
    else if(tAdc.Index == 4)
    {
        tAdc.An4.ValueInstant = ADC1BUF0;
        tAdc.An0.ValueInstant = ADC1BUF1;
        tAdc.An1.ValueInstant = ADC1BUF2;
        tAdc.An2.ValueInstant = ADC1BUF3;

        AD1CHS0 = 0x0003;
        tAdc.Index = 3;
    }
```

然后，下一步又会切换到 Index 是 3 的时候对应的分支。就这样，就是一个轮询切换 AN3 AN4 的过程。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

```
/*==========================================================
static void AdcSample(void)
{
    if(tAdc.Index == 3)
    {
        tAdc.An3.ValueInstant = ADC1BUF0;
        tAdc.An0.ValueInstant = ADC1BUF1;
        tAdc.An1.ValueInstant = ADC1BUF2;
        tAdc.An2.ValueInstant = ADC1BUF3;

        AD1CHS0 = 0x0004;
        tAdc.Index = 4;
    }
    else if(tAdc.Index == 4)
    {
        tAdc.An4.ValueInstant = ADC1BUF0;
        tAdc.An0.ValueInstant = ADC1BUF1;
        tAdc.An1.ValueInstant = ADC1BUF2;
        tAdc.An2.ValueInstant = ADC1BUF3;

        AD1CHS0 = 0x0003;
        tAdc.Index = 3;
    }
    else
    {
        tAdc.An3.ValueInstant = ADC1BUF0;
        tAdc.An0.ValueInstant = ADC1BUF1;
        tAdc.An1.ValueInstant = ADC1BUF2;
        tAdc.An2.ValueInstant = ADC1BUF3;

        AD1CHS0 = 0x0003;
        tAdc.Index = 3;
    }
}
```

而这个 else 分支的话，是初始时刻，Index 是 0 的时候，先执行的分支。后续就会变为 3，然后就在 3 4 之间来回切换。这样的话，我们也就读取到了 ADC 转换的数据。

```
/*==========================================================
void BldcInt(void)
{
    AdcSample();
    BldcStatusControl();
    AdcFilter();
    BldcCount();
}
```

这个函数呢，放在中断入口的时候就读取。也就是说，ADC 中断进来，就先把对应的结果读取到，然后，再执行其它的任务。这是我们获取启动信号的第一步，先把对应旋钮转换的结果给读取到。那么，为了让采样得到的值稍微平滑一些，或者说跳动小一些，我们一般会对采样的实时值做一个滤波。常用的就是均值滤波，所以，在使用前呢，我们一般会先对需要滤波的值做一个滤波处理，然后再使用滤波的值去使用。

```
/*==========================================================
void BldcInt(void)
{
    AdcSample();
    BldcStatusControl();
    AdcFilter();
    BldcCount();
}
```

所以呢，这里也就写了这么一个滤波函数。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

```
/*=====================================================================*/
static void AdcFilter(void)
{
    //Vbus电压 64次平均滤波处理
    tAdc.An3.ValueSum += tAdc.An3.ValueInstant;
    tAdc.An3.ValueNum ++;
    if(tAdc.An3.ValueNum >= 64)
    {
        tAdc.An3.ValueAverage = tAdc.An3.ValueSum >> 6;
        tAdc.An3.ValueSum = 0;
        tAdc.An3.ValueNum = 0;
    }
    //旋钮电压 8次平均滤波处理
    tAdc.An4.ValueSum += tAdc.An4.ValueInstant;
    tAdc.An4.ValueNum ++;
    if(tAdc.An4.ValueNum >= 8)
    {
        tAdc.An4.ValueAverage = tAdc.An4.ValueSum >> 3;
        tAdc.An4.ValueSum = 0;
        tAdc.An4.ValueNum = 0;
    }
}
```

这个函数的实际实现呢，就是这样的，也就是取多次采样的瞬时值进行相加，然后取平均。这里呢，也对 Vbus 采样的值做了一个滤波，目的是为了做过压，欠压保护。Vbus 电压值呢，一般波动的范围很小。也就是说，电压值一般不会发生突变，所以对它的滤波次数呢，可以多一些。而旋钮电压采样值呢，有的时候我们要去拧旋钮，所以也希望这个值有一定的实时性，所以这个值的滤波次数呢，可以小一些。咱们这里呢，电压是 64 次平均，而旋钮电压是 8 次平均。这样的话，就实现了对 Vbus，旋钮电压的均值滤波。好的，后面我们就可以使用这两个滤波值去实现相应的功能了。

```
/*=====================================================================*/
static void AdcPotProcess(void)
{
    unsigned int Temp;

    Temp = tAdc.An4.ValueAverage << 6;

    if(Temp < 655)
        Temp = 0;
    if(Temp > 65535)
        Temp = 65535;

    tBldc.PotValue = Temp;
}
```

咱们来看下这个函数。首先呢，我们的 AD 转换值的范围呢是 0~1023，而矢量圆的半径呢，是 0~1。这个值呢，是归一化的，并且对应转换为了(0.16)格式的数据。所以，矢量圆的半径呢，我们程序中表示就是 0~65535 范围内的数据。我们要做的转换呢，就是把 0~1023 的数据，转换到 0~65535。所以，这段程序中的对旋钮采样的平均值左移 6 位，就是实现这个转换，0~1023 左移 6 位的话，就变到了 0~65472 范围。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

```
static void AdcPotProcess(void)
{
    unsigned int Temp;

    Temp = tAdc.An4.ValueAverage << 6;

    if(Temp < 655)
        Temp = 0;
    if(Temp > 65535)
        Temp = 65535;

    tBldc.PotValue = Temp;
}
```

然后呢，中间有一个限幅的处理。处理完之后呢，就把结果赋值给 tBldc.PotValue 变量了。

```
static void AdcPotProcess(void)
{
    unsigned int Temp;

    Temp = tAdc.An4.ValueAverage << 6;

    if(Temp < 655)
        Temp = 0;
    if(Temp >= 65535)
        Temp = 65535;

    tBldc.PotValue = Temp;
}
```

处理之后呢，这个变量的范围就在 0~65472 以内了。它其实就可以作为矢量圆半径调整的值，作为 SVPWM 模块的给定。但是呢，一般我们会给设定值做一个斜坡处理。也就是说，我快速拧电位器到一个新的位置，给定到 SVPWM 模块的值呢，不会立即变化，而是按照一个增量去变化。这个增量变化相当于让电机的加减速柔和一些。

```
static void PwmDutyControl(void)
{
    unsigned int Temp;

    Temp = tBldc.PwmCount;

    if(Temp == 0)
    {
        tBldc.PwmCount = 10;

        if(tBldc.PotValue > tBldc.DutyInner)
        {
            Temp = tBldc.PotValue - tBldc.DutyInner;
            if(Temp > 100)
                tBldc.DutyInner += 100;
            else if(Temp > 10)
                tBldc.DutyInner += 10;
            else
                tBldc.DutyInner = tBldc.PotValue;
        }
        else
        {
            Temp = tBldc.DutyInner - tBldc.PotValue;
            if(Temp > 100)
                tBldc.DutyInner -= 100;
            else if(Temp > 10)
                tBldc.DutyInner -= 10;
            else
                tBldc.DutyInner = tBldc.PotValue;
        }
    }
}
```


本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

所以，这里就写了这么一个函数来实现这个功能。1023《6 = 65472，这是数据归一化，矢量圆的半径呢，是 0~1 之间。后面不会缩小，相当于电位器的输入在 0-1 之间，电位器充当了调节 M 的角色。

今天内容就到这里，大家有没有问题？

问：如果不柔和会怎样？

答：拧得太快可能响应跟不上，电机的加速度变化过快，di/dt 就会比较大。

问：这个时候是开环控制么？

答：已经有霍尔参与了，是闭环控制。

问：电机的加速度变化有极值限定吗？是怎样限定的？

答：有的，上面的斜坡，就是一种常见的限定方法。

```
if(Temp < 655)
    Temp = 0;
if(Temp >= 65535)
    Temp = 65535;
```

问：为什么小于 655？

答：人为给定的，工程实测选择的。

大家早点休息，晚安！

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。